

Assignment 1 - Named Entity Recognition (NER)

[2026 -NLP] 2026/10/06

Submission Deadline: 2026/10/20 23:59

Task Description

Named entities are phrases that represent real-world objects like persons, organizations, locations, companies, and more. Your task is to identify these entities within unstructured text.

For example: *U.N. official Ekeus heads for Baghdad*

In this example:

- *U.N.* is an **organization**,
- *Ekeus* is a **person**,
- *Baghdad* is a **location**.

This assignment focuses on the **WNUT-2016** dataset, which contains **10 fine-grained NER categories**: **person, geo-location, company, facility, product, music artist, movie, sports team, TV show**, and **other**. This dataset is extracted from English-language tweets, so it presents the additional challenge of informal text.

For NER task, the **BIO tagging scheme** will be used for labeling:

- **B-TYPE**: The first word in an entity (e.g., “**B-person**”).
- **I-TYPE**: The words within an entity (e.g., “**I-person**”).
- **O**: Non-entity words (words that are not part of any entity).

For this assignment, you are provided with **training data**(**train.txt**), **validation data**(**dev.txt**), and **test data** (**test.txt**, without entity tags).

You need to train your model on the **training data**, tune it using the validation data (see how your model performs on **validation data**, and go back to tune the hyperparameters or model architecture). Finally, use your model to predict entity tags for the **test data**, and save the results in **result.txt** following the same format as **dev.txt** for submission. Your predicted results will be evaluated against the ground truth.

Key words: sequence labeling, NER

Requirements

- Only **Python** is allowed.
- You may use any machine learning library such as **TensorFlow**, **Keras**, or **PyTorch**.
- You are free to reference code snippets from open-source projects, tutorials, or blogs, but **do not clone entire projects**. Implement the model independently.
- You must use **sequential models**, such as **RNN**, **GRU**, **LSTM**. It is allowed to modify model architecture or add extra modules to improve model performance.
- You must use **pre-trained word embeddings** like **Word2Vec** or **GloVe** in your model.

Evaluation metrics

We will use the metrics from **CoNLL-2003** to measure performance: **precision**, **recall**, and **F1-score**.

In this context:

- **Precision** is the percentage of correctly identified entities out of all entities your system found.
- **Recall** is the percentage of entities in the dataset that your system correctly identified.
- **Exact-match F1-score** is the harmonic mean of precision and recall.

Your model will be evaluated on its **exact-match F1-score** using the test data.

Grading (Late submission will be -10 points)

- **Source Code Completeness (50%)**
- **F1-Score (20%)**
- **Report (30%)**

Submission Rules

Please follow the format strictly! If you don't follow the format, your score will be -5 points.

Your submission must be packed into a .zip file named **<studentid>_hw1.zip** and uploaded to the platform. The .zip file must include:

1. A Python source code file: **<studentid>.py/.ipynb**.
2. A **result.txt** file containing your predicted named entity tags in the same format as **dev.txt**.
Do not rearrange the lines.
3. A **requirements.txt** file listing the Python libraries used. To generate this file, run the command: `pip freeze > requirements.txt`
4. **<studentid>_report.pdf**. Your report should include details such as data preprocessing, model architecture, training process, evaluation results, and any other relevant observations.